Node.js — Day 25

Backend Organization + Modules + Mini Project

Day 25 Goal
Understand why backend projects are organized into separate files/modules, how **require()** and **module.exports** connect those files, and build a small working Node.js application using command-line input and validation.

1. Why Organize Backend Code?

A very large backend file becomes difficult to understand, maintain, test, and reuse. Separating related code into files gives each part a clearer responsibility.

Simple rule
One file → one main responsibility.
This does not mean one function per file. It means related functionality should stay together.

```
backend/
■
■■■ server.js  → starts/configures the server
■■■ user.js    → user-related functionality
■■■ order.js   → order-related functionality
■■■ utils.js   → reusable helpers
```

2. Backend Code vs Business Logic

Backend code means everything needed for the backend to work. **Business logic** means the application's rules, decisions, and domain-specific behavior.

Example	Business logic?
server.listen(3000)	No — server setup.
require('./user')	No — module loading.
Give 10% discount when total > ■1000	Yes — application rule.
Don't allow checkout when stock = 0	Yes — application rule.

3. CommonJS Modules

Node.js can split an application across multiple files. In the CommonJS module system used in this course, one file can export functionality and another file can require it.

> **Memory model**
> **module.exports → "I want to share this."**
> **require() → "I want to use that."**

```
// user.js
function getUser() {
    return "Bhavana";
}

module.exports = getUser;
```

```
// server.js
const getUser = require("./user");

console.log(getUser());
```

The **./** means the module is a local file relative to the current file. Installed packages such as Express are normally required without **./**.

4. Reusable Helper Functions

```
// utils.js
function calculateTotal(price, quantity) {
    return price * quantity;
}

module.exports = calculateTotal;
```

```
// server.js
const calculateTotal = require("./utils");
console.log(calculateTotal(100, 3));
// 300
```

5. Our Day 25 Mini Project

We built a small command-line User application that combines today's module concepts with earlier concepts such as **process.argv**, **Number()**, and **process.exit()**.

```
user-app/
■■■ app.js
■■■ user.js
■■■ utils.js
```

File	Responsibility
app.js	Gets input, validates it, coordinates modules, and displays results.
user.js	Creates and returns user data.
utils.js	Contains a reusable welcome-message helper.

6. user.js — Creating the User

```
function createUser(name, age) {
    return {
        name: name,
        age: age
    };
}

module.exports = createUser;
```

When **createUser("Bhavana", 26)** is called, it returns an object containing the name and age. The function is exported so **app.js** can use it.

7. app.js — Input, Validation, and Coordination

```
const createUser = require("./user");
const getWelcomeMessage = require("./utils");

const name = process.argv[2];
const age = Number(process.argv[3]);

if (!name || !age) {
    console.log("Please provide name and age");
    process.exit();
}

const user = createUser(name, age);

console.log("User:", user.name);
console.log("Age:", user.age);
console.log("Message:", getWelcomeMessage(user.name));
```

8. Application Flow

```
Command line
   ↓
process.argv
   ↓
app.js
   ↓
Validate input
   ↓
createUser() from user.js
   ↓
User object
   ↓
getWelcomeMessage() from utils.js
   ↓
Display result
```

9. Why Number()?

Command-line arguments arrive as strings. **Number(process.argv[3])** converts the age argument into a number.

Validation

!name means the name is missing/falsy and **!age** means the age value is falsy. **||** means OR. If the condition is true, the program prints an error and exits.

10. Running the Application

```
node app.js Bhavana 26

User: Bhavana
Age: 26
Message: Welcome Bhavana!
```

```
node app.js

Please provide name and age
```

11. What We Practiced

Skill	Status
Multi-file Node.js project	Practiced
module.exports	Practiced
require()	Practiced
process.argv	Practiced
Number()	Practiced
Input validation	Practiced
process.exit()	Practiced
Separation of responsibilities	Practiced

12. Day 25 Assessment

Question	Correct answer
Why separate backend code into different files/modules?	**B** — Keep responsibilities organized and make code easier to maintain/reuse.
What does module.exports = createUser do?	**B** — Makes createUser available to other files.
What is app.js mainly responsible for in our mini project?	**B** — Coordinate input, validation, modules, and output.

Assessment result
3 / 3 correct. Day 25 concepts and the practical mini project were completed.

13. Day 25 Cheat Sheet

Concept	Remember
Backend organization	Separate related responsibilities into appropriate files/modules.
Module	A file that can contain and expose functionality.
module.exports	Exports/shares a value or function.
require()	Imports/loads a module.
./user	Points to a local module/file.
process.argv[2]	First user-provided command-line argument.
Number(process.argv[3])	Converts the third argv item to a number.
process.exit()	Stops the current Node.js process.
Business logic	Application/domain rules and decisions.

Mental model
app.js coordinates → modules provide functionality → each part has a clear responsibility.

Day 25 Complete
You completed backend organization, CommonJS modules, require(), module.exports, command-line input, validation, and a working Node.js mini project.

Long-term learning note: good backend structure is not about creating folders for the sake of it. The goal is to keep responsibilities clear so the application remains understandable and maintainable as it grows.